LAB PRACTICUM: OpenMP Multi-Core Scaling in Python

Completed task sheet with measured results (Numba prange, threading layer: omp)

Student Name:	Abdimurat Dulat	Student ID:	230103296
Hardware / CPU:	Intel64 Family 6 Model 186 (hybrid P/E-core laptop CPU), 12 hardware threads	Date & Section:	2 October 2026 / _____

Table 1. Headline results

Metric	Value
Challenge 1: baseline T_1 (1 thread)	1.9541 s
Challenge 1: T_{max} (12 threads)	0.3722 s (speedup 5.25x, efficiency 43.8%)
Challenge 3: runtime (float64, 12 threads, 300 steps)	0.256 s
Challenge 3: throughput	2638.37 Megacells/s
Whiteboard: best speedup	Task 1: 5.25x at 12 threads; Task 3 (stencil scaling): 2.42x at 8 threads

Challenge 1: Monte Carlo pi and Amdahl's Law

120,000,000 samples; JIT warm-up excluded. Single run per thread count.

Threads	Time (s)	Speedup	Efficiency (%)	Implied serial fraction
1	1.9541	1.00x	100.0	-
2	1.0028	1.95x	97.4	2.6%
4	0.7274	2.69x	67.2	16.3%
8	0.4354	4.49x	56.1	11.2%
12	0.3722	5.25x	43.8	11.7%

A. Baseline and maximum-thread runtime

T_1 = 1.9541 s and T_{max} = 0.3722 s (12 threads); see Table 1.

B. Why the warm-up call is required

The first call to an @njit function triggers type inference and LLVM compilation (and starts the thread pool). Measured: the first call took 0.849 s, the second 0.00016 s (about 5300 times faster). Without the warm-up, compilation would be about 43% of T_1 and would corrupt the baseline and every speedup computed from it.

C. Did efficiency stay at 100%?

No. It fell from 97.4% at 2 threads to 43.8% at 12 threads. If Amdahl's Law with a constant serial fraction were the only cause, the implied serial fraction would be constant; instead it is 2.6% at 2 threads and 11-16% at 4-12 threads, so other effects dominate. Likely factors (the core topology was not verified): (1) the CPU is hybrid, so threads beyond the first two land on SMT siblings or slower E-cores that do less work per thread; (2) prange splits iterations evenly, so threads on fast cores finish early and wait for the slowest one; (3) turbo frequency drops as more cores become active; (4) a small serial part (thread start-up, combining partial sums). Memory-bus contention is not a main factor here because this kernel touches almost no memory; that effect appears in Challenge 3.

D. Why += inside prange does not corrupt the count

Numba recognises a scalar accumulation in a prange loop as a reduction. Each thread gets a private copy of inside_circle initialised to 0, accumulates locally with no sharing, and the private copies are summed after the loop. This is equivalent to OpenMP reduction(+:inside_circle), so there is no data race. (Each thread also has its own random-number state.)

Challenge 2: Load Imbalance and Scheduling (Mandelbrot 2500x2500, max_iter 1000)

Decomposition	Render time	Note
Row-parallel (t_rows)	1.106 s	ran first
Column-parallel (t_cols)	1.075 s	2.8% faster; outputs identical

A. t_rows vs t_cols

Column-parallel finished 0.031 s (2.8%) faster, which is within run-to-run noise: each variant was timed once, and the row version ran first. The honest conclusion is that there is no meaningful difference on this machine. The cost per pixel (up to 1000 iterations) is so large that the memory access pattern is almost completely hidden.

B. Row-major layout and cache lines

In a C-contiguous array, img[r, c] sits at offset (r*W + c)*4 bytes, so neighbouring c values are adjacent and one 64-byte cache line holds 16 int32 values. A thread that walks along a row writes sequentially and uses every cache line fully. A thread that walks down a column writes with a stride of W*4 = 10,000 bytes: every write touches a different cache line (poor spatial locality, extra TLB pressure), and threads handling adjacent columns write into the same cache lines, which can cause false sharing. img[c, r] mirrors this. In this lab the effect is not visible in the timings because computation dominates each write.

C. Load imbalance and dynamic scheduling

Rows near the top lie almost entirely outside the set and escape in 2-3 iterations, so they are cheap. Central rows cross the interior of the set and run to max_iter, so they are very expensive. With static scheduling each thread gets a fixed contiguous block: the thread with the cheap block finishes early and idles at the barrier while the thread with the central rows is still working; total time equals the slowest thread. Dynamic scheduling hands out small chunks of iterations as threads become free, so fast threads take more chunks and all threads finish at about the same time. The cost is some scheduling overhead per chunk, so the chunk size must not be too small.

D. Output image

mandelbrot_output.png is attached separately with this task sheet.

Challenge 3: Stencil Computation and Memory Bandwidth (1500x1500, 300 steps)

Variant (12 threads)	Time (s)	Megacells/s
float64	0.256	2638.37
float32 (dtype changed only, as in the lab)	0.138	4897.69
float32 with float32 constants (extra test)	0.268	2519.97

Threads (float64)	Time (s)	Megacells/s	Speedup	Efficiency (%)	Est. traffic (GB/s)
1	0.621	1086.09	1.00x	100.0	17
2	0.364	1855.99	1.71x	85.3	30
4	0.311	2171.44	2.00x	49.9	35
8	0.257	2629.91	2.42x	30.2	42
12	0.277	2434.79	2.24x	18.7	39

Estimated traffic assumes a minimum of 16 bytes per cell (8 B read + 8 B write); real traffic with write-allocate is up to about 1.5 times higher. These are estimates, not measurements.

A. Runtime and throughput

0.256 s and 2638.37 Megacells/s (float64, 12 threads); see Table 1.

B. float64 to float32

Runtime dropped from 0.256 s to 0.138 s, a factor of 1.86x, close to the 2x expected from halving the bytes moved. Each cell does roughly 7 floating-point operations for about 16 bytes of DRAM traffic (about 0.4 FLOP per byte), so the kernel is limited by memory bandwidth, not by arithmetic. float32 halves the traffic and the array size (18 MB to 9 MB per grid), which can also let more of the data stay in cache. In this variant the constants 0.20 and 4.0 remain float64, so part of the arithmetic is still done in double precision; the near-2x gain therefore comes mostly from memory, not compute. Caveat: this is from a single run, and the same float64 configuration varied by about 8% between runs (0.256 s vs 0.277 s). The extra variant with float32 constants measured 0.268 s, slower than expected; the cause has not been identified and it needs repeated runs.

C. Memory roofline

Attainable performance is min(peak compute, arithmetic intensity x memory bandwidth). The stencil has low arithmetic intensity, so it sits on the bandwidth-limited slope, and the memory bandwidth is shared by all cores through the same memory controllers. A few cores can already saturate it; additional cores only add more requests to the same bus and wait for DRAM. The measured scaling shows this: speedup reaches 2.0x at 4 threads, 2.42x at 8 threads, and then falls slightly to 2.24x at 12 threads, with efficiency collapsing to 18.7%, so doubling the cores does not double the speed.

Measurement notes: laptop on wall power; each configuration was measured in a single run, so values carry roughly 5-10% noise. Threading layer reported by Numba: omp.